Seal

Decentralized Secrets Management for Sui

Table of Contents

1. Introduction — the encryption gap, Seal's solution, and use cases
2. Identity-Based Encryption — IBE foundations, crypto primitives, and architecture
3. Access Policies — identity model, `seal_approve`, patterns, and composition
4. Key Servers & Threshold — threshold trade-offs and Seal's chain of trust
5. Inside the Encryption — encrypt and decrypt flows, session keys
6. Move Integration — `seal_approve` in detail, the TypeScript SDK, and full stack
7. User Flow — end-to-end walkthrough and verification model
8. Developer Workflow — getting started and key server operations
9. Security & Limitations
10. Key Takeaways + Resources

The Encryption Gap

Blockchains solve **authentication** — proving who you are and what you own. But they lack native **encryption** — controlling who can read data.

Aspect	Authentication	Encryption
Status	Solved	Missing
Mechanism	Signatures, zkLogin, Passkeys	No standard
Controls	Who can write/transfer	Who can read
Tooling	Wallets, SDKs, standards	Ad hoc, centralized

Today's workarounds: ephemeral keys shared via side channels, one-off KMS systems, centralized services that defeat the purpose of decentralization.

> How do we get programmable encryption without centralized trust?

Seal: The Solution

Programmable encryption with on-chain access policies and decentralized key servers.

Programmable

Access policies are **Move smart contracts** you control. Define who can decrypt what — and when — using the full expressiveness of Move.

Decentralized

Threshold encryption across multiple independent key servers. No single server can break privacy. Choose your own trust set.

Client-Side

Data is encrypted and decrypted **locally**. Key servers derive identity keys on demand but never see your plaintext.

Real World Use Cases

Gated Content & Subscriptions

Encrypt premium content on Walrus. Subscribers get time-limited decryption rights enforced by Move. Expiry is automatic — no re-encryption needed.

Secure Voting & MEV Protection

Keep ballots and orders encrypted until a future timestamp. Prevents frontrunning and premature reveals. Tally on-chain after the deadline.

Private Storage & Messaging

Encrypt personal data to your own address. End-to-end messaging using Sui addresses as identities — no key exchange protocol needed.

What Is Identity-Based Encryption?

A cryptographic scheme where **any string** serves as a public key — no key exchange needed.

> A mailbox anyone can drop letters into, but only the named recipient can open — and the recipient doesn't need to exist yet when you send the letter.

Four operations define IBE:

```
Setup(λ) → (mpk, msk) | Encrypt(mpk, id, msg) → ciphertext | Derive(msk, id) → sk | Decrypt(sk, ct) → msg
```

- **Setup** — Generate a master public key (`mpk`) and master secret key (`msk`)
- **Encrypt** — Anyone encrypts using `mpk` + an identity string. No contact with key holder.
- **Derive** — The master key holder derives a secret key for any identity on demand
- **Decrypt** — The derived secret key decrypts the ciphertext

Cryptographic Building Blocks

Component	Primitive	Role
KEM	Boneh-Franklin IBE over BLS12-381	Threshold key encapsulation — encrypts symmetric key shares
DEM (fast)	AES-256-GCM	Client-side data encryption — recommended for most use cases
DEM (on-chain)	HMAC-based CTR mode	Symmetric encryption that can be decrypted in Move
Key transport	ElGamal-encrypted BLS	Protects derived keys in transit from key server to client
Threshold	Shamir's Secret Sharing	Splits symmetric key so any `t`-of-`n` servers can reconstruct

Key insight: The KEM/DEM structure means threshold operations only touch small key material. Bulk data encryption uses efficient symmetric crypto — the expensive IBE math is limited to a tiny key, not your data.

Seal's Architecture: Two Pillars

On-Chain Pillar

Access policies live in **Move smart contracts**. `seal_approve*` functions define who can decrypt. Evaluated **read-only** via `dry_run` — no gas, no state changes.
The Move package address (`PackageId`) acts as the **identity namespace** — it controls all IBE identities that start with `[PackageId]`.

Off-Chain Pillar

Key servers hold **IBE master secret keys**. They are stateless — only `msk` + a Sui full node connection. Derive identity keys on demand. Horizontally scalable.
Philosophy: "The less a key server stores, the easier it is to secure."

```
Move Package (access policy) ←── IBE Identity: [PackageId][id] ──→ Key Server (key derivation)
```

The IBE Identity Model

Every encrypted object targets a specific IBE identity:

Identity = `[PackageId] || [id]`

- `PackageId` — Namespace isolation. The Move package address. Different packages have completely separate identity spaces.
- `id` — Policy-specific identifier. An address, a timestamp, an object ID — any bytes the policy needs.

> PackageId is the zip code (which post office handles it); id is the specific mailbox.

Properties: deterministic, unbounded domain, no interaction needed to construct.

Pattern	What id Encodes
Private data	`bcs::to_bytes(owner_address)`
Time-lock	`bcs::to_bytes(unlock_timestamp)`
Allowlist	`bcs::to_bytes(list_object_id)`
Subscription	`bcs::to_bytes(service_id)`
Voting	`bcs::to_bytes(vote_id)`

The `seal_approve` Interface

Move entry functions that act as **gatekeepers** — if they don't abort, the key server releases the decryption key.

> A bouncer who checks credentials using rules the club owner wrote — can't change the club, only decides who gets in.

Five conventions:

1. **Name prefix** — Must start with `seal_approve` (e.g., `seal_approve`, `seal_approve_with_nft`)
2. **First parameter** — `id: vector<u8>` — the requested identity (without package prefix)
3. **Non-public entry** — Declared as `entry fun` (not `public`) for upgradeability
4. **Abort on denial** — Successful return = access approved. Abort = access denied.
5. **Side-effect free** — Evaluated via `dry_run` — read-only, no state changes

```
entry fun seal_approve(id: vector<u8>, ctx: &TxContext) {
    let caller_bytes = bcs::to_bytes(&ctx.sender());
    assert!(id == caller_bytes, ENoAccess);
}
```

Five Access Control Patterns

Pattern	Identity Encodes	Use Case	Access Model
Private Data	Owner address	Personal storage, private NFTs	Single owner; transfer moves rights
Allowlist	List object ID	Partner data rooms, gated drops	Shared object; update without re-encrypting
Subscription	Service ID	Premium content, paid APIs	Time-limited pass; auto-expires
Time-Lock	Unlock timestamp	Voting, auctions, MEV protection	No access before; open after deadline
Secure Voting	Vote ID	DAO governance, sealed bids	Encrypted ballots; threshold decryption for tally

These are reference implementations — starting points you can customize and extend. Since access logic is just Move code, the combinatorial space is unbounded.

Composing Patterns

Since these are **standard Move functions**, you can combine them freely.

Example: Pre-signed URL = time check + allowlist check

```
entry fun seal_approve(
    id: vector<u8>,
    list: &Allowlist,
    c: &clock::Clock,
    ctx: &TxContext,
) {
    let mut prepared: BCS = bcs::new(id);
    let expiry = prepared.peel_u64();
    assert!(c.timestamp_ms() <= expiry, EExpired);
    assert!(allowlist::contains(list, ctx.sender()), ENoAccess);
}
```

Key insight: Composition is free because access logic is just Move code. No special framework, no plugin system — combine time checks, ownership checks, token balances, oracle data, or any on-chain state into a single gatekeeper function.

Key Servers & Threshold Encryption

A **key server** is a stateless service: IBE master secret key + Sui full node connection. Two endpoints:

- `/v1/service` — Server info (object ID, public key, URL)
- `/v1/fetch_key` — Key derivation (evaluates policy, returns derived key)

> Independent notaries — you need any 2 of 3 to sign off. No single notary can forge alone; if one is on vacation, the other two suffice.

Threshold Trade-offs

Config	Privacy	Liveness
1-of-1	None	None
2-of-3	Tolerates 1	Tolerates 1
3-of-5	Tolerates 2	Tolerates 2

Privacy = compromised servers tolerated
Liveness = unavailable servers tolerated

2-of-3 is the recommended default.

Seal's Chain of Trust

Seal Trusts

- **Key server integrity** — threshold subset is honest
- **Full node integrity** — accurate chain state for policy evaluation
- **Correct access policy** — your Move code expresses your intended rules

"Your Move code IS the security policy. If `seal_approve` is wrong, Seal faithfully enforces the wrong rules."

Seal Does NOT Trust

- **Storage layer** — encrypted data is safe at rest regardless
- **Network transport** — derived keys are encrypted under ephemeral keys
- **dApp code** — session keys limit exposure to a single package + TTL

```
User Wallet → Session Key → dApp → PTB → Key Server → Full Node (dry run) → Approve / Deny
```

Inside the Encryption: The 7-Step Flow

Encryption is **entirely local** — no key server contact needed.

> Put a letter in a safe, then cut the safe key into pieces and put each piece in a different locked mailbox.

1. Policy
Choose package + construct identity `id`

→

2. Servers
Select n key servers + threshold t

→

3. k_sym
Generate random symmetric key

→

4. AES
Encrypt data with k_sym (AES-256-GCM)

→

5. Shamir
Split k_sym into n shares (threshold t)

→

6. IBE
Encrypt each share under server's `mpk`

→

7. Bundle
Package into `EncryptedObject`

The Decryption Process

Decryption **requires** key server interaction and policy evaluation.

1. Build PTB
Construct transaction calling `seal_approve*` with identity + required objects (Clock, NFTs, etc.)

→

2. Request Keys
SDK sends PTB to ≥t key servers. Each server dry-runs the PTB on its full node.

→

3. Derive & Return
If policy approves, each server returns its derived key — encrypted under the requester's ephemeral key.

→

4. Reconstruct & Decrypt
SDK combines t derived keys → reconstructs k_sym → AES-GCM decrypts the data.

Why ephemeral encryption matters: Derived keys in transit are encrypted to the requester only. Even if the transport is intercepted, the attacker cannot use the keys. Only the session that requested them can decrypt.

Session Keys: The User Consent Layer

Problem: Decryption keys are returned to the dApp's web page. How do we ensure user consent?

Solution: Time-limited, package-scoped session keys.

> A visitor badge at a secure building — gets you through doors in one department for a limited time, then expires.

```
const sessionKey = await SessionKey.create({
  address: suiAddress,
  packageId: fromHEX(packageId),
  ttlMin: 10,                    // expires in 10 minutes
  suiClient,
});

const message = sessionKey.getPersonalMessage();
const { signature } = await wallet.signPersonalMessage(message);
sessionKey.setPersonalMessageSignature(signature);
// Session key is now active — reuse for multiple decryptions
```

Properties: Scoped to one package. Expires automatically. Storable in `localStorage` / `IndexedDB` for cross-tab persistence.

Move Integration: `seal_approve` in Detail

Three elements a developer writes in Move:

1. **The `seal_approve*` function** — gatekeeper logic
2. **Identity construction** — BCS encoding of policy conditions
3. **Shared objects** — Allowlists, Subscriptions, or other state the policy reads

Canonical example: Time-lock encryption

```
entry fun seal_approve(id: vector<u8>, c: &clock::Clock) {
    let mut prepared: BCS = bcs::new(id);    // Decode the identity bytes
    let t = prepared.peel_u64();              // Extract the unlock timestamp
    let leftovers = prepared.into_remainder_bytes();
    assert!(
        (leftovers.length() == 0) &&          // No extra bytes (strict parsing)
        (c.timestamp_ms() >= t),              // Current time >= unlock time
        ENoAccess,
    );
}
```

The `Clock` object is passed by the key server in the PTB. The `id` is `bcs::to_bytes(timestamp)` — constructed by the encrypting client.

The TypeScript SDK

`@mysten/seal` — the primary interface for encryption and decryption.

Encrypt (local)

```
const {
  encryptedObject,
  key: backupKey,
} = await client.encrypt({
  threshold: 2,
  packageId: fromHEX(packageId),
  id: fromHEX(id),
  data,
});
```

No key server contact. Returns encrypted bytes + optional backup key for disaster recovery.

Decrypt (interactive)

```
const tx = new Transaction();
tx.moveCall({
  target: `${pkg}::mod::seal_approve`,
  arguments: [
    tx.pure.vector("u8", fromHEX(id)),
  ],
});
const txBytes = await tx.build({
  client: suiClient,
  onlyTransactionKind: true,
});

const decrypted = await client.decrypt({
  data: encryptedObject,
  sessionKey,
  txBytes,
});
```

Putting It Together

Application Stack

Move Package — Access policy (`seal_approve*`)
TypeScript dApp — Encrypt/decrypt via `@mysten/seal`
Key Servers — Derive identity keys on demand
Sui Blockchain — Hosts policies, objects, identity model

Envelope Encryption

For large data: encrypt content with your own AES key, then encrypt that key with Seal.

Benefits:

- Rotate servers without re-encrypting content
- Decouple storage from key management
- Only the small symmetric key touches Seal's threshold system

Batch Decryption

For galleries, feeds, or multi-file scenarios: use `fetchKeys()` with a multi-command PTB.

```
await client.fetchKeys({
  ids: [id1, id2, id3],
  txBytes,
  sessionKey,
  threshold: 2,
});
```

Groups requests to reduce key server round trips.

User Flow Overview

Encryption (local)

1. Developer deploys Move package with `seal_approve*`
2. App selects key servers + threshold
3. SDK encrypts **locally** using server public keys
4. Store ciphertext anywhere — Walrus, Sui, S3, IPFS

No arrows to key servers. Encryption is purely local.

Decryption (interactive)

1. User authorizes session key (wallet signature)
2. dApp builds PTB calling `seal_approve*`
3. SDK contacts ≥t key servers
4. Each server dry-runs PTB on full node
5. Servers return derived keys (encrypted to requester)
6. SDK reconstructs k_sym + decrypts

Multi-party exchange — but transparent to the user.

What Gets Verified Where

At encryption time: No verification — encryption is a local operation using server public keys from the chain.

At decryption time:

Check	Verified By
Session key signature is valid	Key server
PTB calls only `seal_approve*` functions	Key server
`seal_approve*` doesn't abort (policy passes)	Full node (dry run)
Derived key encrypted to requester only	Key server (ephemeral key)
Threshold reconstruction is consistent	SDK (client-side)
AES-GCM decryption succeeds (integrity check)	SDK (client-side)

Every actor in the chain verifies only what it is responsible for. No single entity sees the full picture — this is separation of concerns by design.

Getting Started

Four steps from zero to decrypting your first secret on testnet.

1. Install the SDK

`npm install @mysten/seal`

2. Choose Key Servers

Select from verified testnet providers. Six servers available with open access, operated by independent organizations.

3. Write & Deploy Access Policy

Write a `seal_approve*` function in Move. Build with `sui move build`. Publish with `sui client publish`.

4. Encrypt & Decrypt

`client.encrypt()` to seal data (local). `client.decrypt()` to retrieve it (interactive with key servers).

Key Server Operations & Providers

Open vs Permissioned

Aspect	Open	Permissioned
Access	Any package	Allowlisted only
Master key	Single key	Per-client derived
Isolation	Shared	Full per-client
Use case	Public / testing	B2B / commercial
Key export	N/A	Supported

Verified Testnet Providers

Provider	Operator
testnet-1	Mysten Labs
testnet-2	Mysten Labs
Ruby Nodes	Independent
NodeInfra	Independent
Studio Mirai	Independent
Overclock	Independent

Servers register on-chain `KeyServer` objects. The SDK discovers them by object ID. Infrastructure: horizontal scaling, full node dependency, master key in KMS/vault.

Security & Limitations

Protects Against

- **Unauthorized decryption** — threshold policies + Move access control
- **Data at rest** — client-side AES-256-GCM
- **Key server collusion** — fewer than t compromised = safe
- **Transport interception** — ephemeral key encryption
- **Non-malleability** — ciphertexts cannot be transformed into valid related ciphertexts

Does NOT Protect Against

- **Buggy access policies** — code review is on you
- **Decryption key leaks** — client-side risk after decryption
- **All t+ servers compromised** — threshold is the trust bound
- **Metadata / size leakage** — Seal does not conceal message sizes
- **Server set is fixed** — cannot change servers for existing ciphertexts
- **No on-chain audit trail** — key delivery isn't logged to chain

Key Takeaways

IBE removes key exchange — encrypt to any identity string, no recipient interaction needed
Move IS the policy — transparent, on-chain, composable access control you write yourself
Threshold distributes trust — no single key server can break privacy
Client-side encryption — key servers never see your data
Trust is configurable — choose your servers, your threshold, your policies

SDK — `npm install @mysten/seal` · **Docs** — seal-docs.wal.app · **GitHub** — github.com/MystenLabs/seal · **Discord** — Seal channel on Sui Discord